IO INCIDENTOPS

IncidentOps

Cloud Incident & Service Reliability Management Platform

DEVOPS CAPSTONE PROJECT

Antara Sanjay Utane

ANTARAUTANE11@GMAIL.COM

github.com/Aana-1025/incidentops-devops-capstone

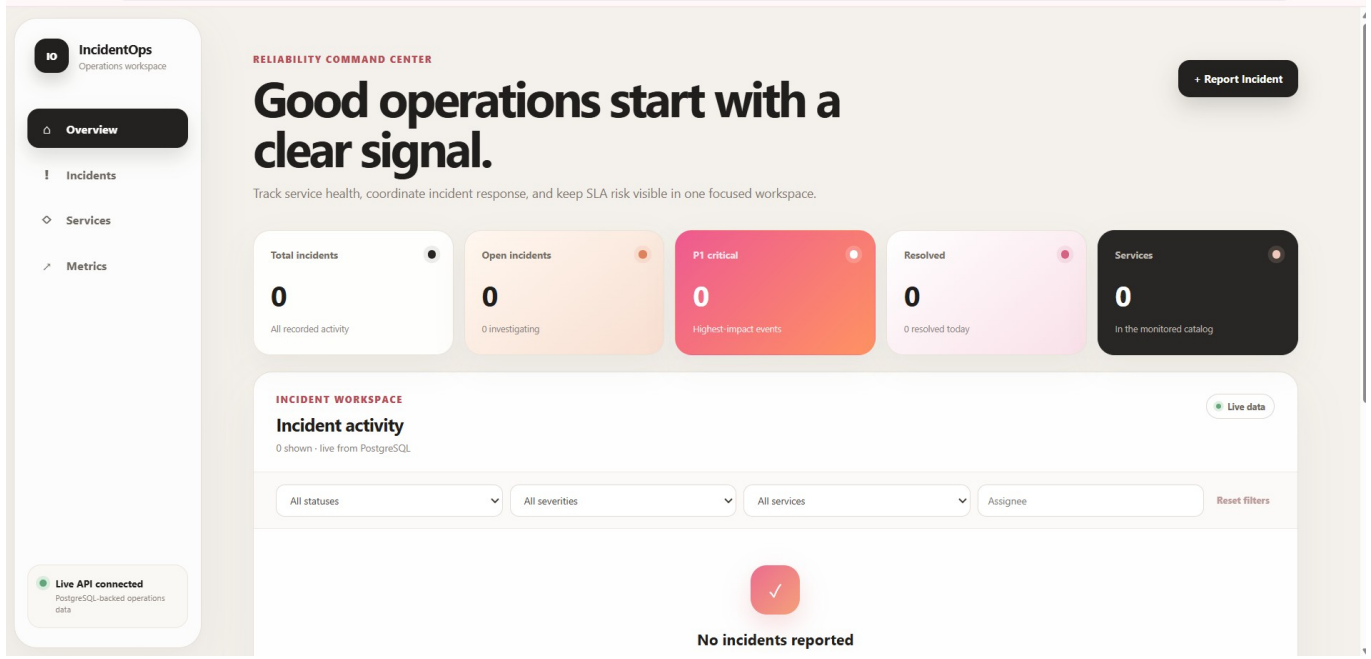
FROM CODE TO OPERATIONS

- React + FastAPI + PostgreSQL
- Docker + GitHub Actions
- Trivy + GHCR
- Terraform + AWS EKS
- Kubernetes + Helm + HPA
- Prometheus + Grafana

Reliable delivery • observable runtime • reversible infrastructure

What I built

A small operations application with a complete end-to-end delivery path.



IncidentOps dashboard

Core capabilities

- Register monitored services and report incidents
- Track severity, lifecycle status, assignee and SLA risk
- Persist operational data in PostgreSQL through FastAPI
- Expose Swagger, liveness, readiness and Prometheus metrics

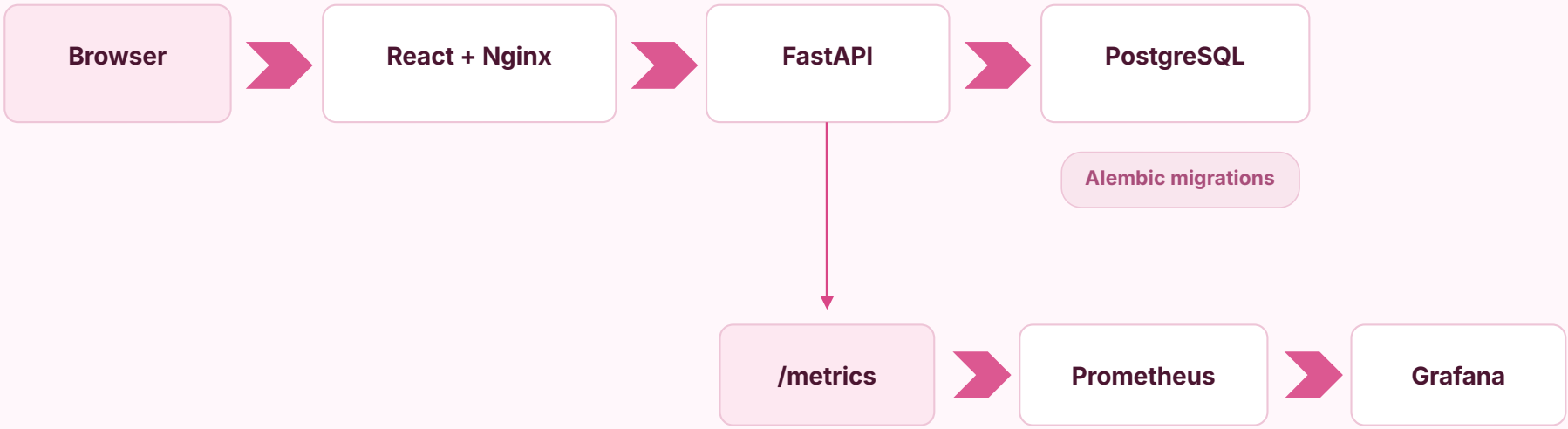
SLA policy

P1	30 min
P2	2 h
P3	8 h
P4	24 h

Architecture overview

Application flow, delivery flow and observability are connected but independently verifiable.

RUNTIME



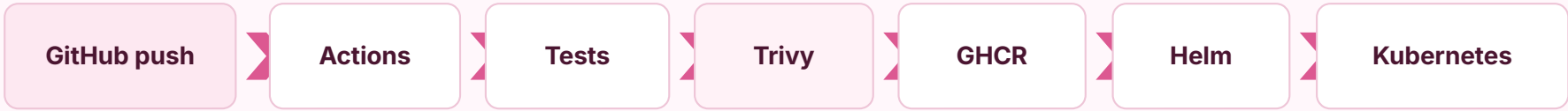
Infrastructure

Terraform

AWS VPC + EKS
2 public subnets
EBS CSI + Pod Identity

Local: Docker Desktop
Cloud: AWS EKS

DELIVERY



Quality first: tests + containers

I verify behavior before packaging the application for delivery.

```
(.venv) PS C:\Users\hp\Desktop\IncidentOps\backend> python -m pytest -v
============================= test session starts =============================
platform win32 -- Python 3.11.9, pytest-9.1.1, pluggy-1.7.0 -- C:\Users\hp\Desktop\IncidentOps\backend\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\hp\Desktop\IncidentOps\backend
configfile: pytest.ini
testpaths: tests
plugins: anyio-4.15.1
collected 16 items

tests/test_incidents.py::test_create_list_get_and_filter_incident PASSED [  6%]
tests/test_incidents.py::test_resolve_and_reopen_incident_updates_timestamp PASSED [ 12%]
tests/test_incidents.py::test_delete_incident_removes_it PASSED [ 18%]
tests/test_incidents.py::test_invalid_incident_input_returns_validation_error PASSED [ 25%]
tests/test_incidents.py::test_missing_incident_and_service_references_return_not_found PASSED [ 31%]
tests/test_incidents.py::test_service_with_incident_cannot_be_deleted PASSED [ 37%]
tests/test_incidents.py::test_incident_statistics_are_database_derived PASSED [ 43%]
tests/test_services.py::test_create_list_and_get_service PASSED [ 50%]
tests/test_services.py::test_update_service_preserves_omitted_fields PASSED [ 56%]
tests/test_services.py::test_duplicate_service_name_returns_conflict PASSED [ 62%]
tests/test_services.py::test_invalid_service_input_returns_validation_error PASSED [ 68%]
tests/test_services.py::test_missing_service_returns_not_found PASSED [ 75%]
tests/test_services.py::test_delete_unused_service_removes_it PASSED [ 81%]
tests/test_sla.py::test_sla_policy_calculates_all_statuses_deterministically PASSED [ 87%]
tests/test_system.py::test_health_endpoint_reports_process_liveness PASSED [ 93%]
tests/test_system.py::test_readiness_endpoint_queries_test_database PASSED [100%]

============================= 16 passed in 3.61s ==============================
(.venv) PS C:\Users\hp\Desktop\IncidentOps\backend>
```

Pytest - 16 passed

```
PS C:\Users\hp\Desktop\IncidentOps> docker compose ps
NAME                    IMAGE                      COMMAND                  SERVICE    CREATED       STATUS                   PORTS
incidentops-backend-1   incidentops-backend:local  "/usr/local/bin/inci…"   backend    4 hours ago   Up 4 hours (healthy)     0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
incidentops-frontend-1  incidentops-frontend:local "/docker-entrypoint.…"   frontend   4 hours ago   Up 4 hours (healthy)     0.0.0.0:3000->8080/tcp, [::]:3000->8080/tcp
incidentops-postgres-1  postgres:17-alpine         "docker-entrypoint.s…"   postgres   5 hours ago   Up 5 hours (healthy)     0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp
PS C:\Users\hp\Desktop\IncidentOps>
```

Docker Compose - healthy stack

```
PS C:\Users\hp\Desktop\IncidentOps> docker compose exec backend id
uid=10001(incidentops) gid=10001(incidentops) groups=10001(incidentops)
PS C:\Users\hp\Desktop\IncidentOps> docker compose exec frontend id
uid=101(nginx) gid=101(nginx) groups=101(nginx)
```

Non-root runtime users

16 backend tests

3 healthy Compose services

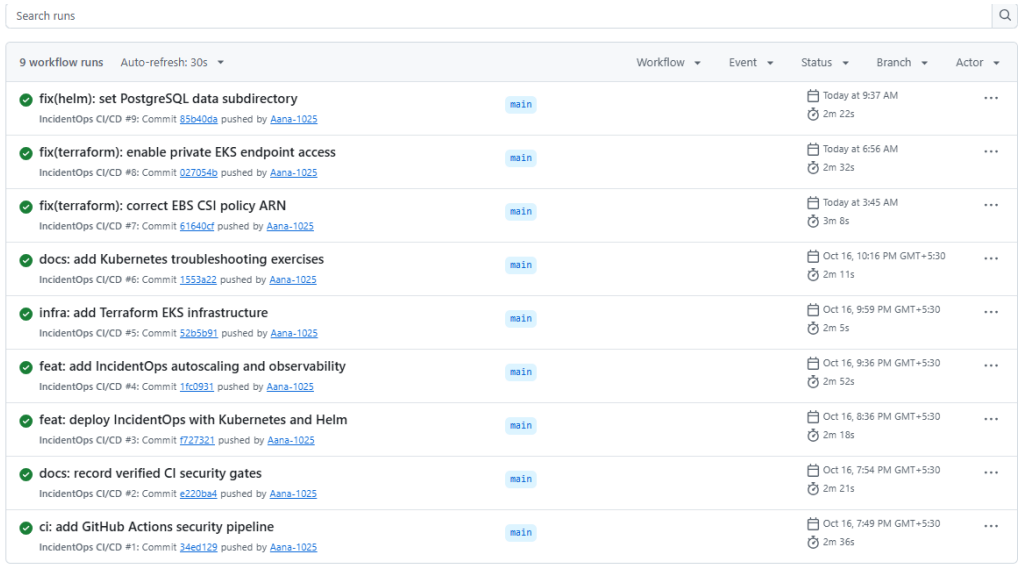
10001 backend runtime UID

101 frontend Nginx UID

Multi-stage frontend image keeps Node tooling out of the runtime image.

CI/CD with a security gate

Only tested and scanned images are published to GHCR.



GitHub Actions - successful runs



Trivy - HIGH / CRITICAL gate

Push → **Pytest + build** → **Trivy** → **GHCR** → **SHA-tagged images**

M10 release: 2b76fa3...

Terraform on AWS: a timeboxed EKS environment

The infrastructure was created for evidence, verified, then completely destroyed.

AWS architecture

VPC 10.0.0.0/16

Public subnet ap-south-1a

Public subnet ap-south-1b

Amazon EKS 1.36

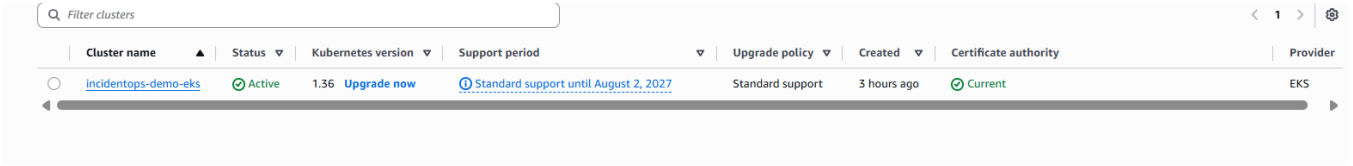
1 × t3.medium managed worker
EBS CSI + Pod Identity
No NAT Gateway • No RDS • No AWS Load Balancer

Cost-aware design

Endpoint /32 restricted

```
PS C:\Users\hp\Desktop\IncidentOps> Get-Content "C:\Users\hp\Desktop\IncidentOps-Evidence\terraform-original-plan-output
.txt"
Plan: 21 to add, 0 change, 0 to destroy.
PS C:\Users\hp\Desktop\IncidentOps>
```

Terraform plan: 21 add, 0 change, 0 destroy



AWS EKS cluster - ACTIVE

```
PS C:\Users\hp\Desktop\IncidentOps> kubectl get nodes -o wide
NAME                                          STATUS   ROLES    AGE    VERSION               INTERNAL-IP   EXTERNAL-IP    OS-IMAGE                         KERNEL
-VERSION                        CONTAINER-RUNTIME
ip-10-0-2-4.ap-south-1.compute.internal       Ready    <none>   116m   v1.36.4-eks-b9510e3   10.0.2.4      3.110.156.94   Amazon Linux 2023.12.20261012   6.18.5
4-121.166.amzn2023.x86_64 (amd64)   containerd://2.3.5+unknown
PS C:\Users\hp\Desktop\IncidentOps>
```

kubectl - worker node Ready

Kubernetes + Helm: reproducible application delivery

Helm packages the complete runtime instead of relying on manual cluster edits.

```
PS C:\Users\hp\Desktop\IncidentOps> kubectl get pods -n incidentops -o wide
NAME                                  READY   STATUS      RESTARTS   AGE     IP           NODE                                           NOMINATED NODE   READIN
ESS GATES
incidentops-backend-57cbf96f65-ljnt5  1/1     Running     0          4m28s   10.0.2.228   ip-10-0-2-4.ap-south-1.compute.internal   <none>           <none>
incidentops-backend-57cbf96f65-zlfs5  1/1     Running     0          3m57s   10.0.2.236   ip-10-0-2-4.ap-south-1.compute.internal   <none>           <none>
incidentops-frontend-9cdc98946-fhj5m  1/1     Running     0          26m     10.0.2.203   ip-10-0-2-4.ap-south-1.compute.internal   <none>           <none>
incidentops-frontend-9cdc98946-xjqzq  1/1     Running     0          26m     10.0.2.243   ip-10-0-2-4.ap-south-1.compute.internal   <none>           <none>
incidentops-migrate-2-zszvs           0/1     Completed   0          4m28s   10.0.2.5     ip-10-0-2-4.ap-south-1.compute.internal   <none>           <none>
incidentops-postgresql-0              1/1     Running     0          4m27s   10.0.2.89    ip-10-0-2-4.ap-south-1.compute.internal   <none>           <none>
PS C:\Users\hp\Desktop\IncidentOps>
```

AWS EKS - application pods running

```
PS C:\Users\hp\Desktop\IncidentOps> helm list -n incidentops
NAME            NAMESPACE       REVISION        UPDATED                                 STATUS          CHART                   APP VERSION
incidentops     incidentops     2               2026-10-17 09:38:02.6553579 +0530 IST   deployed        incidentops-0.1.0       0.2.0
PS C:\Users\hp\Desktop\IncidentOps>
```

Helm release deployed

Same chart verified locally and on AWS EKS.

Helm release includes

- 2 frontend replicas + 2 backend replicas
- PostgreSQL StatefulSet + 1 GiB PVC
- Alembic migration Job
- ClusterIP Services + NGINX Ingress
- Backend HPA + ServiceMonitor

/ → frontend **/api → backend**

Autoscaling under load

The backend scales based on CPU demand and returns to a safe baseline.

```
PS C:\Users\hp> kubectl get hpa -n incidentops
NAME                 REFERENCE                       TARGETS       MINPODS   MAXPODS   REPLICAS   AGE
incidentops-backend   Deployment/incidentops-backend   cpu: 4%/50%   2         5         2          18h
PS C:\Users\hp>
```

Before load - 2 replicas

```
PS C:\Users\hp> kubectl get hpa -n incidentops -w
NAME                 REFERENCE                       TARGETS       MINPODS   MAXPODS   REPLICAS   AGE
incidentops-backend   Deployment/incidentops-backend   cpu: 5%/50%   2         5         2          18h
incidentops-backend   Deployment/incidentops-backend   cpu: 482%/50%   2         5         2          18h
incidentops-backend   Deployment/incidentops-backend   cpu: 420%/50%   2         5         4          18h
incidentops-backend   Deployment/incidentops-backend   cpu: 323%/50%   2         5         5          18h
```

During load - scaled up

2 → 4 → 5

backend replicas

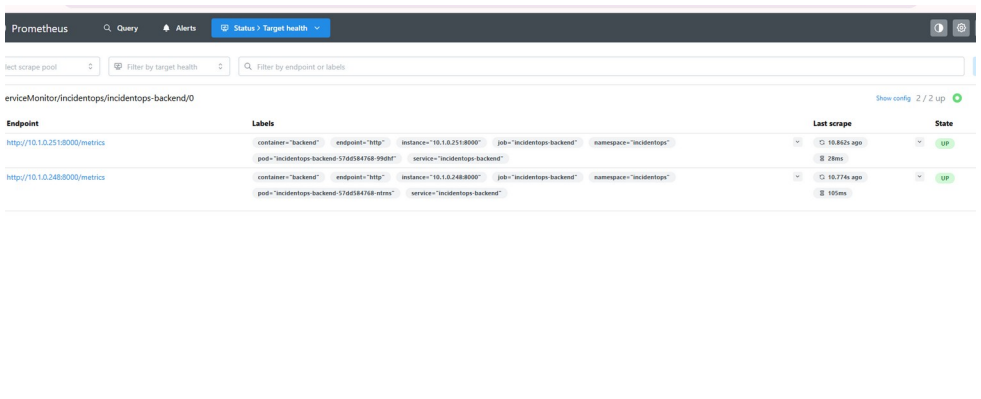
HPA policy

min: 2 max: 5
CPU target: 50%
Metrics Server supplies utilization data

resources.requests configured

Observability: Prometheus + Grafana

Application metrics are scraped from FastAPI and turned into operational dashboards.



Prometheus targets - backend UP



Grafana - IncidentOps observability

2 / 2
backend targets UP

/metrics
FastAPI scrape endpoint

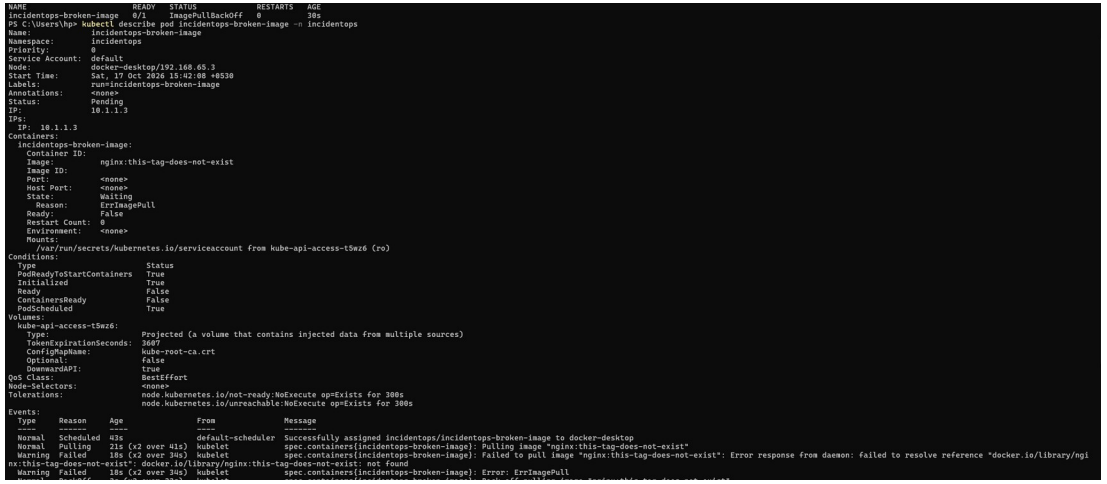
p95
latency panel

rate
request-rate panel

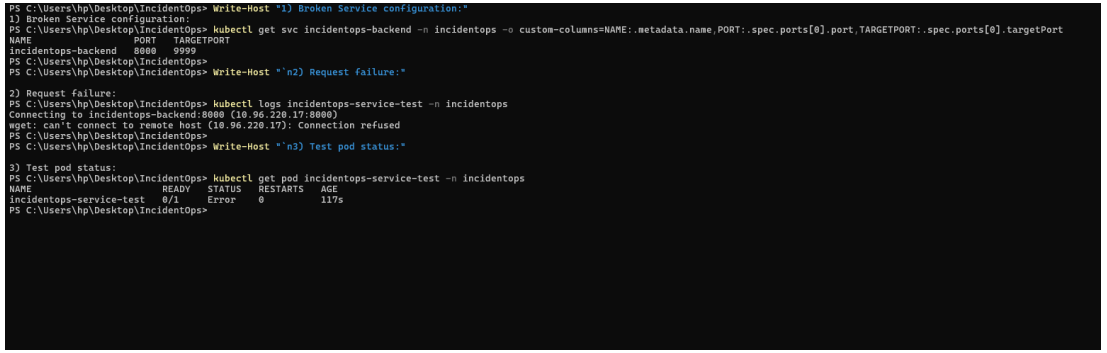
ServiceMonitor connects the backend Service to Prometheus discovery.

Troubleshooting: fail safely, diagnose, recover

I reproduced two common Kubernetes failures without modifying the Helm-managed application.



ImagePullBackOff - invalid image tag



Wrong targetPort - Service connection refused

1. Inspect Pod status + events
2. Trace Service → endpoint → target port
3. Apply correction
4. Verify healthy recovery

Final deployment proof + complete cleanup

The last visible change flowed through CI to a real Helm upgrade, and the AWS environment was torn down.

M10 release path

SLA UI change → Git push → CI + Trivy → GHCR → Helm rev. 6

2b76fa3c3b1c580fe48d4b3b5504c4206793061d

backend SHA verified | frontend SHA verified | browser update visible

Local Docker Desktop Kubernetes was used for the final M10 deployment - AWS was not recreated.



Terraform destroy proof

0	0	21
added	changed	destroyed

No IncidentOps AWS environment remains running.

IO INCIDENTOPS

Ready for the live walkthrough

Codebase → CI/CD → Terraform/AWS → Kubernetes/Helm → Prometheus → Grafana

16	2 + 2	2 → 5	21
tests	app replicas	HPA range	AWS resources destroyed

Antara Sanjay Utane

ANTARAUTANE11@GMAIL.COM

github.com/Aana-1025/incidentops-devops-capstone

Thank you

Next: live evidence